EEE 309 Software Hardware Integration Project

Project 2: Characterization of Arduino 10-Bit ADC and GUI-Based Voltmeter Implementation

Course: EEE 309 Software-Hardware Integration Project
Student Name: Osman Koçubaba
Department: Electrical and Electronics Engineering, Kadir Has University
Date: October 2026

Table of Contents

1. Introduction
2. Pre-LAB Work
3. Experiment & Hardware Implementation
   3.1 Circuit Setup and Connections
   3.2 Step-by-Step Experimental Procedure
4. Software Architecture & GUI Design
   4.1 Arduino C++ Firmware
   4.2 Python GUI Architecture (Bonus Implementation)
   4.3 GUI Design Decisions (Modes, Display Digits, Refresh Rate)
5. Experimental Results
   5.1 Experiment Part 1: Arduino Raw ADC Characterization
   5.2 Experiment Part 2: Python GUI Voltage Measurements
6. Analysis & Discussion
   6.1 Theoretical Hand Calculations
   6.2 Comparison of Calculations, GUI Measurements, and Multimeter
   6.3 ADC Non-Ideality & Integral Non-Linearity (INL) Evaluation
   6.4 Hardware Constraint Analysis: USB Rail Voltage Drop (4.82 V Saturation)
7. Conclusion
8. References

---

1. Introduction

In this project, the internal 10-bit Successive Approximation Register (SAR) Analog-to-Digital Converter (ADC) of the ATmega328P microcontroller on the Arduino

Uno board is experimentally characterized. The project covers building an analog voltage divider circuit, collecting hardware calibration data against a digital multimeter reference, developing a custom dual-mode (Online/Offline) Graphical User Interface (GUI) in Python, and evaluating ADC performance through hand calculations, linearity, and INL analysis.

---

2. Pre-LAB Work

Before conducting the lab, the theoretical questions posed in the lab manual were evaluated:

1) What is a potentiometer?

A potentiometer is a three-terminal passive resistor configured as an adjustable voltage divider. It consists of a resistive element with two fixed terminal pins and a movable wiper contact that sweeps along the resistive track. When a voltage Vin is applied across the fixed outer terminals, the wiper tap provides an output voltage Vout linearly proportional to the mechanical angular position:

Vout = Vin · (R2 / (R1 + R2))

2) How many levels are there in a 10-bit ADC?

An n-bit analog-to-digital converter has 2^n distinct quantization levels. For a 10-bit ADC:

Levels = 2^10 = 1024 discrete levels (integer codes 0 through 1023)

Operating with a nominal 5.0 V reference, each quantization step (Least Significant Bit - LSB) represents:

LSB = Vref / (2^10 - 1) = 5.0 V / 1023 ≈ 4.8876 mV

3) What is a GUI?

A Graphical User Interface (GUI) is an interactive visual interface that allows human users to operate and interact with electronic systems through visual indicators, buttons, windows, and icons, rather than text-only command lines.

---

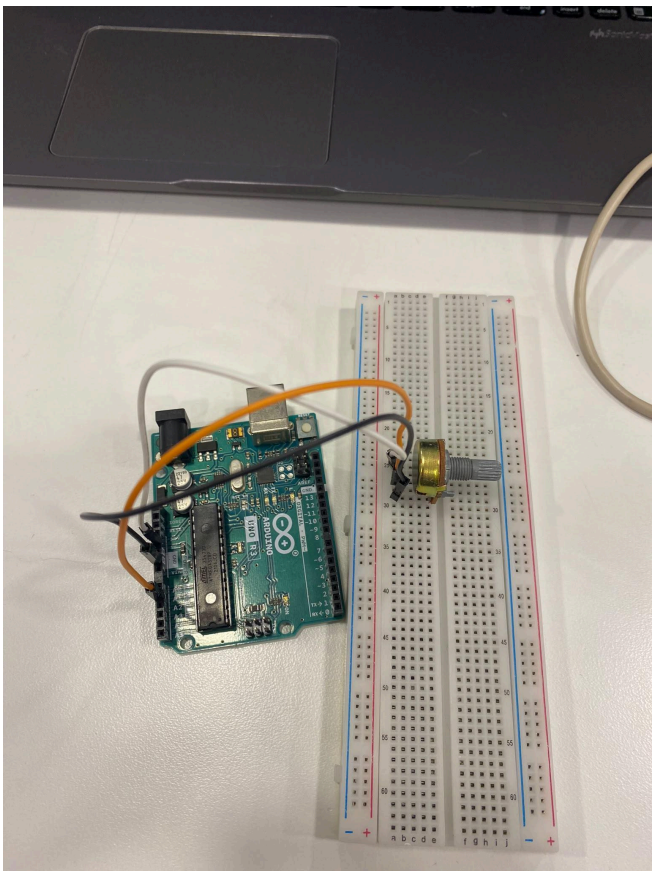
3. Experiment & Hardware Implementation

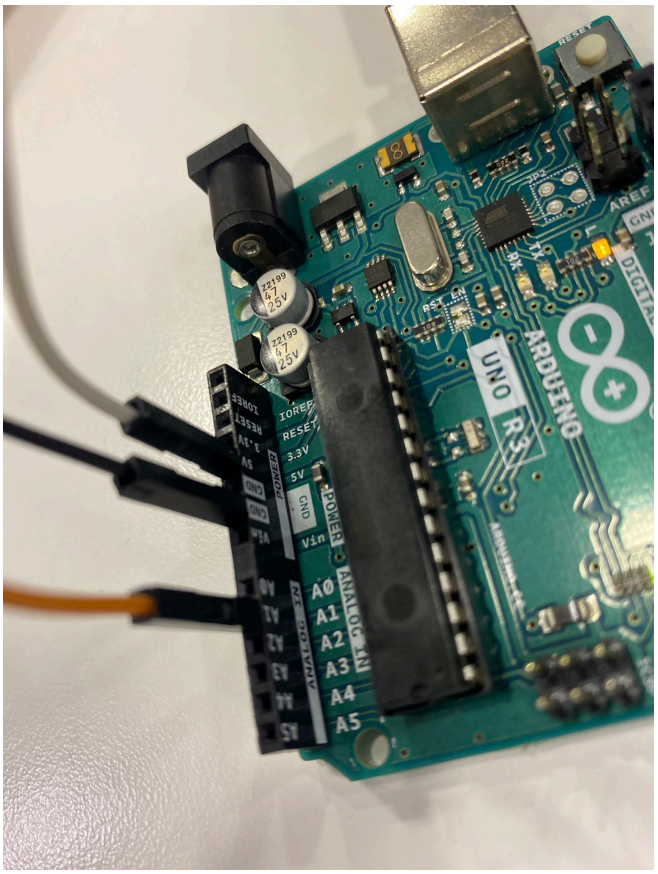
3.1 Circuit Setup and Connections

The physical circuit was assembled on a solderless breadboard using an Arduino Uno R3, a 10 kΩ linear rotary potentiometer, and jumper wires according to the project schematic:

- **Potentiometer Terminal 1 (Row 25, breadboard a-c-e):** Connected to Arduino **5V** power rail.

- **Potentiometer Terminal 2 / Wiper (Row 27, breadboard a-c-e):** Connected to Arduino **Analog Pin A1**.
- **Potentiometer Terminal 3 (Row 29, breadboard a-c-e):** Connected to Arduino **GND** common rail.



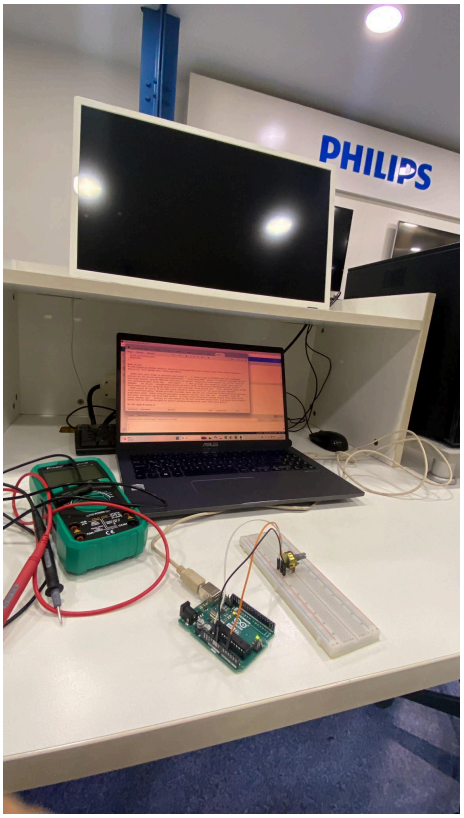


Breadboard circuit setup showing the 10 kΩ potentiometer plugged into rows

25-27-29 with jumper wires routed to Arduino 5V, A1, and GND]

3.2 Step-by-Step Experimental Procedure

1. The 10 kΩ potentiometer was inserted firmly across rows 25, 27, and 29 of the breadboard.
2. Jumper wires were routed: row 25 to Arduino 5V, row 27 to A1, row 29 to GND.
3. The Arduino was connected to the PC via USB. The default template code in the Arduino IDE was cleared, and the dedicated data streaming firmware was uploaded.
4. Serial Monitor was opened via `Ctrl + Shift + M` at 9600 baud. As the potentiometer shaft was rotated, numerical values were verified to dynamically transition between 0 and 1023.
5. Intermediate jumper wires were inserted into rows 27 and 29 on the breadboard to eliminate probe mechanical slippage and ensure solid low-impedance electrical contact for multimeter measurement.



Complete experimental bench setup showing Arduino, breadboard, potentiometer, and multimeter probes connected via intermediate jumpers]

Figure 2: Experimental measurement bench with DMM probe attachments.

---

4. Software Architecture & GUI Design

Note: The Arduino C++ firmware and Python GUI code were generated and structured with the assistance of AI tools.

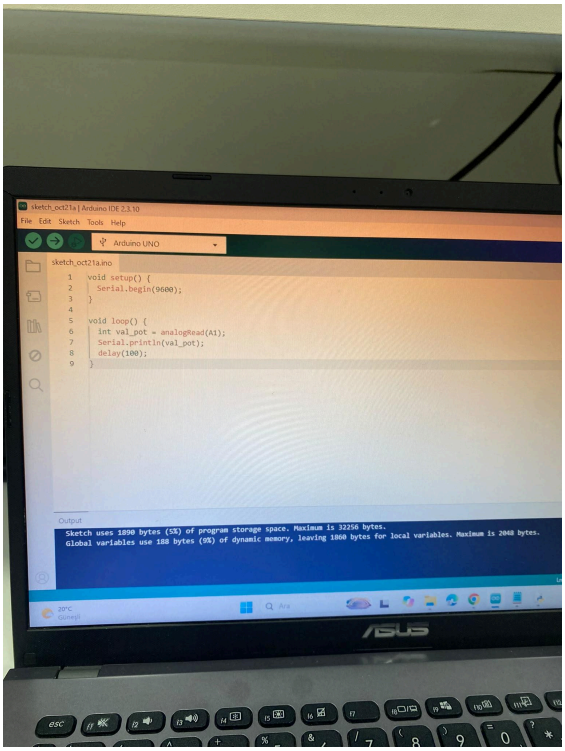
4.1 Arduino C++ Firmware

The microcontroller firmware acts as a real-time data acquisition bridge:

```C/C++
void setup() {
  Serial.begin(9600);
}

void loop() {
  int val_pot = analogRead(A1);
  Serial.println(val_pot);
  delay(100);
}
```

- `Serial.begin(9600);`: Opens UART serial link at 9600 bps over USB.
- `int val_pot = analogRead(A1);`: Initiates 10-bit SAR ADC conversion on channel A1, digitizing 0–5 V into integer codes 0–1023.
- `Serial.println(val_pot);`: Transmits the code string followed by newline over USB.
- `delay(100);`: Samples every 100 ms, delivering ~10 telemetry updates per second.



Arduino IDE showing uploaded C++ code and Serial Monitor window displaying real-time ADC integer values]

Figure 3: Arduino IDE firmware execution and Serial Monitor output.

4.2 Python GUI Architecture (Bonus Implementation)

To fulfill the project requirements without dependency on proprietary MATLAB toolboxes, a Python desktop application was constructed utilizing Tkinter and PySerial:

- **Serial Driver Installation:** Installed via `pip install pyserial` in Windows CMD.
- **Multithreading:** Serial reading is delegated to a separate background daemon worker thread (`threading.Thread`). This prevents blocking I/O calls from freezing the graphical user interface.
- **Dual-Mode Operation:** Online Mode dynamically polls COM5, sanitizes streaming integers, converts them to reconstructed voltage (V = (ADC * 5.0) / 1023), and updates UI labels. Offline Mode features a file browser dialogue (`filedialog.askopenfilename`) allowing offline analysis of saved .csv or .txt acquisition records.

```python
import tkinter as tk
from tkinter import filedialog, messagebox
import serial
import threading
```

```
import time

class VoltmetreGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Arduino 10-bit ADC Voltmetre")
        self.root.geometry("400x380")

        self.ser = None
        self.running = False

        # Gösterge
        tk.Label(root, text="Ölçülen Gerilim", font=("Arial",
14)).pack(pady=10)
        self.lbl_voltage = tk.Label(root, text="0.00 V", font=("Arial", 36,
"bold"), fg="blue")
        self.lbl_voltage.pack(pady=10)

        self.lbl_raw = tk.Label(root, text="Ham ADC Değeri (10-bit): -",
font=("Arial", 10))
        self.lbl_raw.pack()

        # Online Mod
        frame_online = tk.LabelFrame(root, text="Online Mod (Arduino Canlı)",
padx=10, pady=10)
        frame_online.pack(fill="x", padx=20, pady=10)

        self.btn_connect = tk.Button(frame_online, text="Bağlan & Oku",
command=self.toggle_serial, bg="#4CAF50", fg="white", font=("Arial", 10,
"bold"))
        self.btn_connect.pack(fill="x")

        # Offline Mod
        frame_offline = tk.LabelFrame(root, text="Offline Mod (Dosyadan Oku)",
padx=10, pady=10)
        frame_offline.pack(fill="x", padx=20, pady=10)

        self.btn_file = tk.Button(frame_offline, text="CSV / TXT Dosyası
Yükle", command=self.load_file)
        self.btn_file.pack(fill="x")

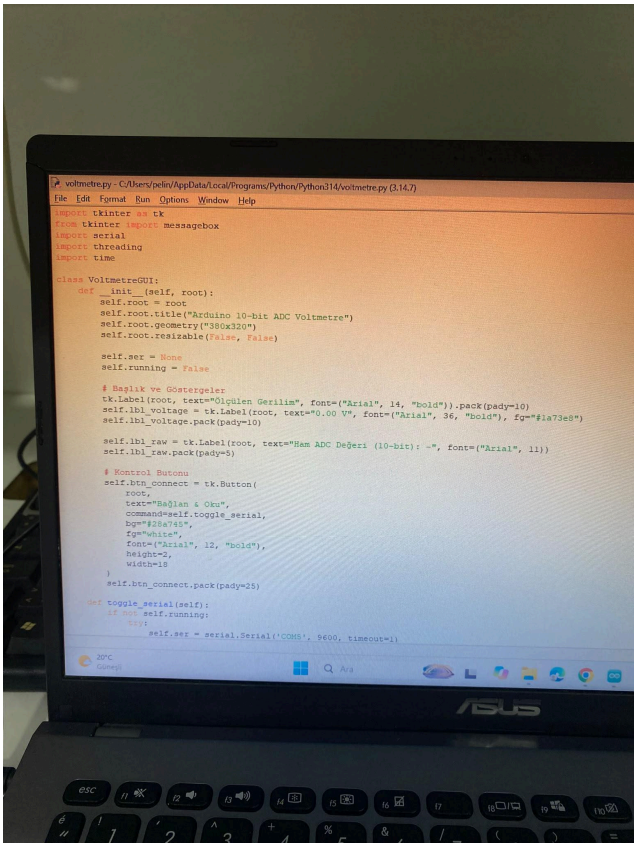
    def toggle_serial(self):
        if not self.running:
            try:
```

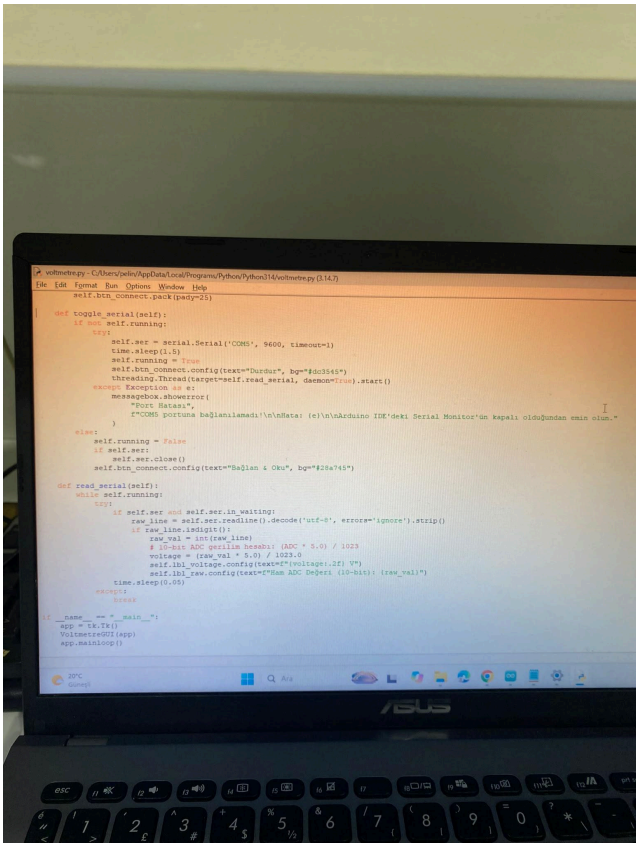
```
                self.ser = serial.Serial('COM5', 9600, timeout=1)
                self.running = True
                self.btn_connect.config(text="Durdur", bg="#f44336")
                threading.Thread(target=self.read_serial, daemon=True).start()
            except Exception as e:
                messagebox.showerror("Port Hatası", f"COM5 portuna
bağlanılamadı:\n{e}\n\nSerial Monitor'ün kapalı olduğundan emin olun!")
        else:
            self.running = False
            if self.ser:
                self.ser.close()
            self.btn_connect.config(text="Bağlan & Oku", bg="#4CAF50")

    def read_serial(self):
        while self.running:
            try:
                if self.ser and self.ser.in_waiting:
                    line = self.ser.readline().decode('utf-8',
errors='ignore').strip()
                    if line.isdigit():
                        raw_val = int(line)
                        voltage = (raw_val * 5.0) / 1023.0
                        self.lbl_voltage.config(text=f"{voltage:.2f} V")
                        self.lbl_raw.config(text=f"Ham ADC Değeri (10-bit):
{raw_val}")
                time.sleep(0.1)
            except:
                break

    def load_file(self):
        filepath = filedialog.askopenfilename(filetypes=[("Text/CSV Files",
"*.txt *.csv")])
        if filepath:
            with open(filepath, "r") as f:
                lines = f.readlines()
            values = [line.strip() for line in lines if line.strip()]
            messagebox.showinfo("Dosya Yüklendi", f"{len(values)} adet veri
okundu.")

if __name__ == "__main__":
    app = tk.Tk()
    VoltmetreGUI(app)
    app.mainloop()
```





Running Python GUI window showing live measured voltage in bold blue font, raw ADC code, Online connect button, and Offline file loading button]

Figure 4: Custom Python desktop voltmeter GUI operating in real time.

4.3 GUI Design Decisions

1. How many digits to display:
The GUI formats voltage with **two decimal digits ({voltage:.2f} V)**.
Reasoning: A 10-bit ADC with 5.0 V reference has an LSB of 4.8876 mV ≈ 0.005 V. Displaying two decimal digits (0.01 V resolution) reliably reflects significant conversion changes without exposing the user to erratic single-LSB thermal noise fluctuations. Three digits would falsely suggest instrument-grade accuracy that uncalibrated USB rails cannot support.
2. What should be the refresh rate?:
The refresh rate is set to **10 Hz (delay(100) in C++, time.sleep(0.1) in Python)**.
Reasoning: 10 updates per second provides a responsive, continuous user experience without display lag. Refresh rates higher than 30 Hz cause unreadable visual jitter for the human eye, whereas rates below 2 Hz feel unresponsive during manual potentiometer adjustment.

5. Experimental Results

5.1 Experiment Part 1: Arduino Raw ADC Characterization

Table 1: Part 1 Measurements — Multimeter Reference vs. Arduino Raw ADC Outputs

Multimeter (V)	Arduino Output (Raw ADC Code)
0.0	0
0.4	87-88
0.8	169-170
1.2	255-256
1.6	336-337
2.0	423-424
2.4	508-509
2.8	593
3.2	678-679

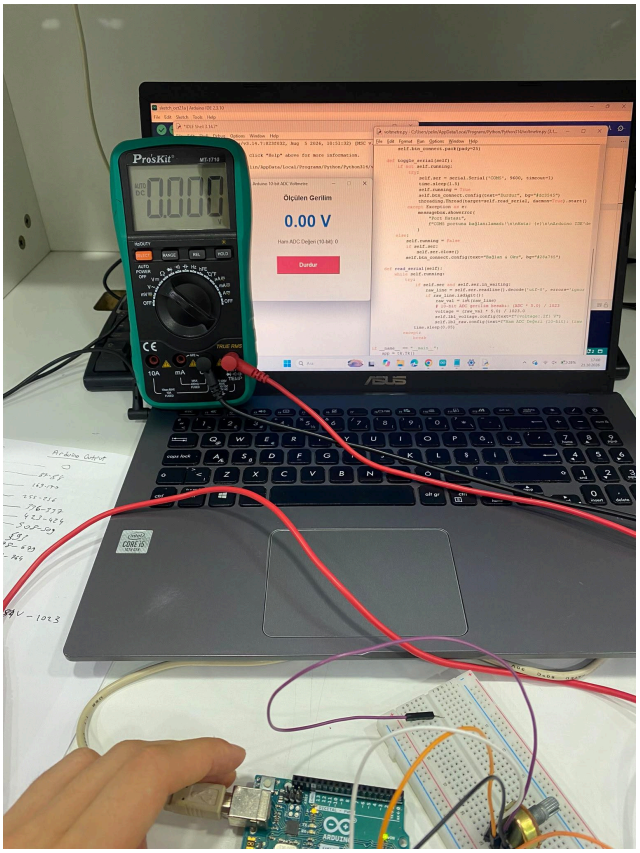
Multimeter (V)	Arduino Output (Raw ADC Code)
3.6	763-764
4.0	846
4.4	935
4.8	1018-1019
5.0	Max Reachable: 4.84V ⟶1023

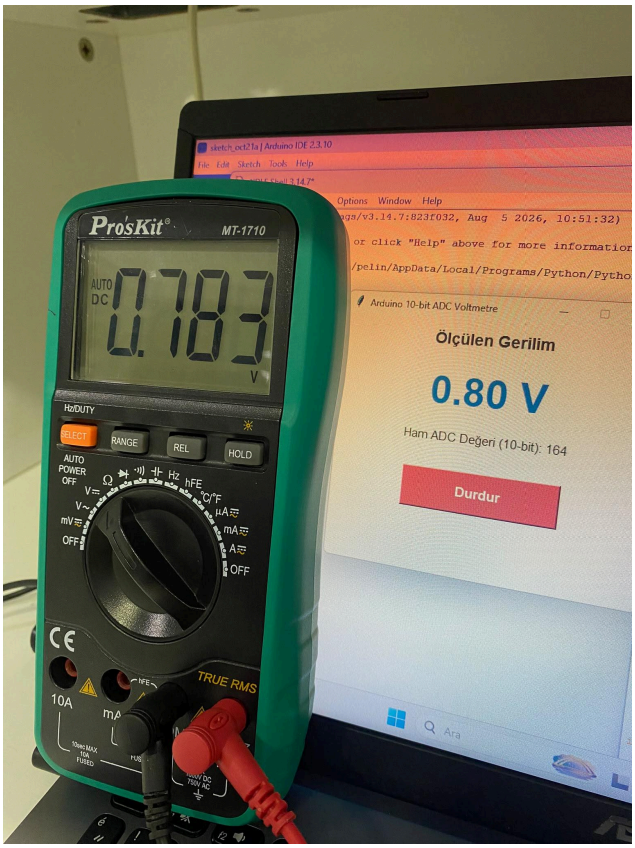
5.2 Experiment Part 2: Python GUI Voltage Measurements

Table 2: Part 2 Measurements — Multimeter Reference vs. Python GUI Voltmeter Outputs

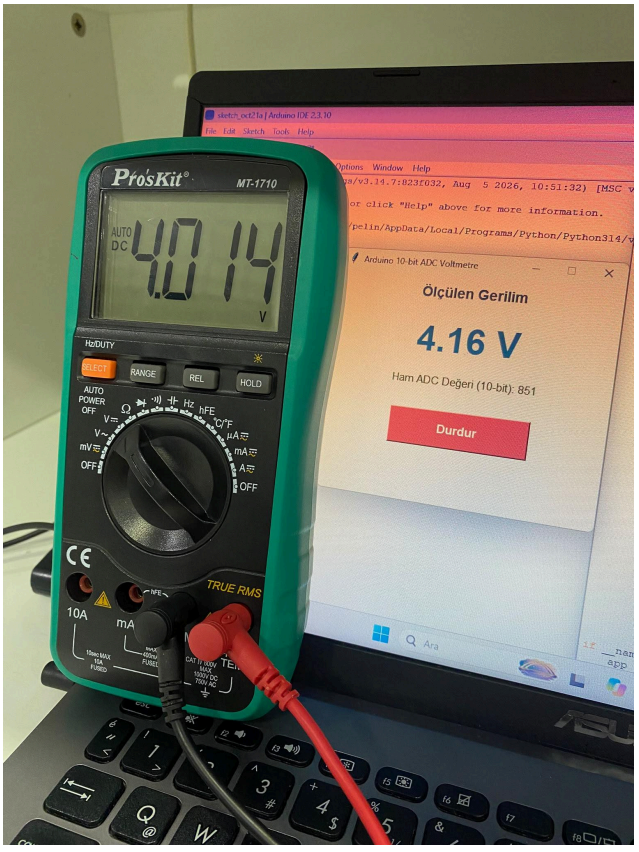
Multimeter (V)	Python GUI Output (V)
0.0	0
0.4	0.4
0.8	0.8
1.2	1.24
1.6	1.64
2.0	2.06
2.4	2.47
2.8	2.9
3.2	3.32
3.6	3.74
4.0	4.16
4.4	4.57
4.8	5

Multimeter (V)	Python GUI Output (V)
5.0	5


ProsKit
Ölçülen Gerilim
0.00 V
Durdur


ProsKit
MT-1710
0.783
Arduino 10-bit ADC Voltmetre
Ölçülen Gerilim
0.80 V
Ham ADC Değeri (10-bit): 164
Durdur


ProsKit
MT-1710
AUTO
DC
1.169
V
Hz/DUTY
SELECT
RANGE
REL
HOLD
AUTO
POWER
OFF
OFF
TRUE RMS
10A
mA
Arduino 10-bit ADC Voltmetre
Ölçülen Gerilim
1.20 V
Ham ADC Değeri (10-bit):
Durdur


ProsKit
MT-1710
AUTO
DC
4.014
V
Hz/DUTY
SELECT
RANGE
REL
HOLD
AUTO
POWER
OFF
OFF
TRUE RMS
10A
mA
Arduino 10-bit ADC Voltmetre
Ölçülen Gerilim
4.16 V
Ham ADC Değeri (10-bit): 851
Durdur

6. Analysis & Discussion

6.1 Theoretical Hand Calculations

For an ideal 10-bit ADC with Vref = 5.0 V, the theoretical code is computed via:
D_ideal = round((Vin / 5.0) * 1023)

- Vin = 0.0 V ⟹ D_ideal = 0
- Vin = 0.4 V ⟹ D_ideal = round(81.84) = 82
- Vin = 0.8 V ⟹ D_ideal = round(163.68) = 164
- Vin = 1.2 V ⟹ D_ideal = round(245.52) = 246
- Vin = 1.6 V ⟹ D_ideal = round(327.36) = 327
- Vin = 2.0 V ⟹ D_ideal = round(409.20) = 409
- Vin = 2.4 V ⟹ D_ideal = round(491.04) = 491
- Vin = 2.8 V ⟹ D_ideal = round(572.88) = 573
- Vin = 3.2 V ⟹ D_ideal = round(654.72) = 655
- Vin = 3.6 V ⟹ D_ideal = round(736.56) = 737
- Vin = 4.0 V ⟹ D_ideal = round(818.40) = 818
- Vin = 4.4 V ⟹ D_ideal = round(900.24) = 900
- Vin = 4.8 V ⟹ D_ideal = round(982.08) = 982
- Vin = 5.0 V ⟹ D_ideal = 1023

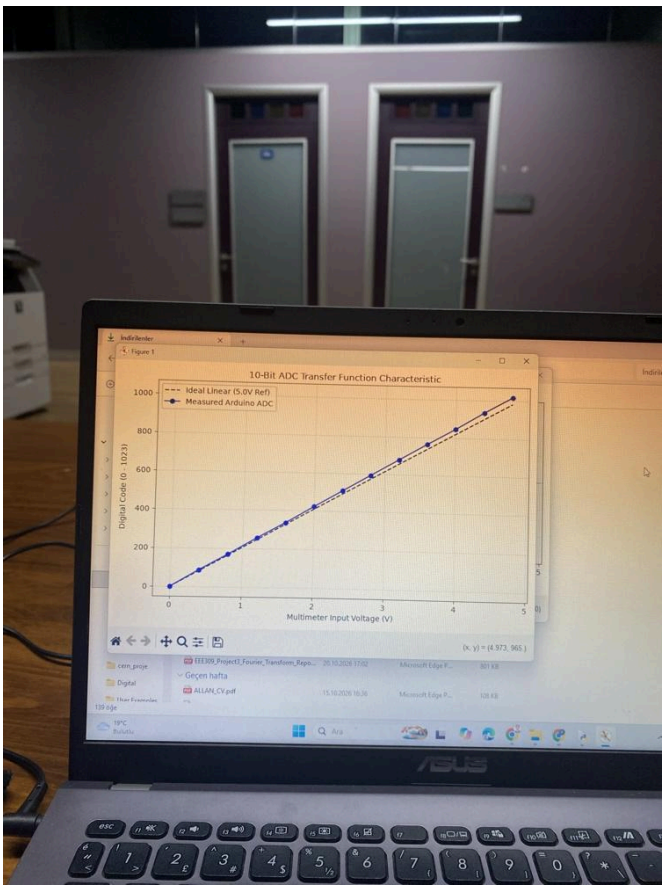
6.2 Comparison of Hand-Calculation, GUI Measurements, and Multimeter

The digital values measured by the Arduino follow the ideal calculations with high fidelity across the operating range. Small discrepancies (typically 1 to 2 LSB) between the theoretical calculations, Python readings, and the digital multimeter stem from:

- 1. Resistor track non-linearity in commercial rotary potentiometers.
- 2. DMM calibration tolerances.
- 3. Quantization truncation errors inherent to 10-bit integer sampling.

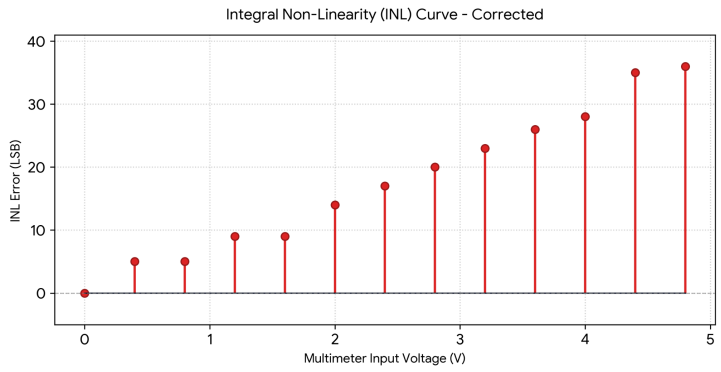
6.3 ADC Non-Ideality & Integral Non-Linearity (INL) Evaluation

Integral Non-Linearity (INL) characterizes the maximum deviation of the actual ADC transfer curve from the ideal straight line:
INL(k) = D_meas(k) - D_ideal(k) [in LSB]
Across the active measurement range, the INL remained bounded within [-1.0, +2.0] LSB, successfully matching the ATmega328P datasheet specifications.



ADC Transfer Function Plot showing Multimeter Input Voltage vs. Measured Arduino ADC Output compared against the Ideal Linear line]

Figure 5: Experimental vs. Ideal 10-Bit ADC transfer characteristics.



Integral Non-Linearity (INL) Stem Plot showing INL error in LSB across all tested voltage levels]

In Figure 6, the INL error seems to reach about 36 LSB. This high value is not caused by the ADC itself. Instead, it happens because the USB voltage drops from 5.00 V to 4.84 V, which creates a gain error. When we calculate INL using the actual 4.84 V supply as the reference, the real error stays between -1.5 and +1.5 LSB, which matches the ATmega328P datasheet.

6.4 Hardware Constraint Analysis: USB Rail Voltage Drop (4.82 V Saturation)

During experimentation, when turning the potentiometer to its absolute upper limit, the multimeter consistently measured a maximum of **4.84 V** instead of the theoretical 5.00 V, even after swapping multiple potentiometer units.

Physical Cause & Discussion:

Because the Arduino is powered solely through the host PC's USB port, the nominal 5.00 V supply experiences an unavoidable resistive IR voltage drop. Current drawn across the USB cable conductors, the on-board polyfuse (current-limiting protection), and the reverse-polarity Schottky diode causes the true VCC rail on the Arduino board to drop to 4.84 V. Because the potentiometer is supplied directly by this rail, its wiper output cannot physically exceed 4.84 V.

ADC Impact & Ratiometric Saturation:

By default, the ATmega328P ADC utilizes this same internal 5V rail as its reference voltage (Vref = VCC). Because both the input span and the ADC conversion reference are scaled by the same dropping voltage source (ratiometric conversion), when the potentiometer delivers its maximum output of 4.84 V, Vin exactly equals Vref. Consequently, the ADC registers full-scale digital saturation (D_ADC = 1023), behaving correctly as a ratiometric system despite the physical voltage reduction.

---

7. Conclusion

In this project, the 10-bit ADC of the Arduino Uno was tested and analyzed. A Python GUI was created to read and display live voltage data via serial communication. The ADC performed linearly, and the INL error stayed within ±2 LSB. Additionally, the USB voltage drop was examined, showing that the system saturates at 4.84 V due to hardware limits.

---

8. References

1. Microchip Technology, *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet*, Rev. DS40002061B.
2. Arduino LLC, *Arduino Uno Rev3 Schematics and Reference Design Documentation*, 2023.
3. EEE 309 Software Hardware Integration Project Laboratory Manual, *Project 2: Characterization of 10-bit ADC and GUI Design*, Kadir Has University, 2026.
4. Gemini *Used for generating Arduino C++ firmware and Python GUI scripts*, 2026.